

Relationships in class diagrams

In UML, a relationship is a connection between model elements. A UML relationship is a type of model element that adds semantics to a model by defining the structure and behavior between model elements.

UML relationships are grouped into the following categories:

Category	Function
Activity edges	Represent the flow between activities
Associations	Indicate that instances of one model element are connected to instances of another model element
Dependencies	Indicate that a change to one model element can affect another model element
Generalizations	Indicate that one model element is a specialization of another model element
Realizations	Indicate that one model element provides a specification that another model element implements
Transitions	Represent changes in state

You can set properties and use keywords to create variations of these relationships.

Relationships in class diagrams show the interaction between classes and classifiers. Such relationships indicate the classifiers that are associated with each other, those that are generalizations and realizations, and those that have dependencies on other classes and classifiers.

The following topics describe the relationships that you can use in class diagrams:

- **Abstraction relationships**
An abstraction relationship is a dependency between model elements that represents the same concept at different levels of abstraction or from different viewpoints. You can add abstraction relationships to a model in several diagrams, including use-case, class, and component diagrams.
- **Aggregation relationships**
In UML models, an aggregation relationship shows a classifier as a part of or subordinate to another classifier.
- **Association relationships**
In UML models, an association is a relationship between two classifiers, such as classes or use cases, that describes the reasons for the relationship and the rules that govern the relationship.
- **Association classes**
In UML diagrams, an association class is a class that is part of an association relationship between two other classes.
- **Binding relationships**
In UML models, a binding relationship is a relationship that assigns values to template parameters and generates a new model element from the template.
- **Composition association relationships**
A composition association relationship represents a whole–part relationship and is a form of aggregation. A composition association relationship specifies that the lifetime of the part classifier is dependent on the lifetime of the whole classifier.
- **Dependency relationships**
In UML, a dependency relationship is a relationship in which one element, the client, uses or depends on another element, the supplier. You can use dependency relationships in class diagrams, component diagrams, deployment diagrams, and use-case diagrams to indicate that a change to the supplier might require a change to the client.

- **Directed association relationships**
In UML models, directed association relationships are associations that are navigable in only one direction.
- **Element import relationships**
In UML diagrams, an element import relationship identifies a model element in another package, and allows the element in the other package to be referenced by using its name without a qualifier.
- **Generalization relationships**
In UML modeling, a generalization relationship is a relationship in which one model element (the child) is based on another model element (the parent). Generalization relationships are used in class, component, deployment, and use-case diagrams to indicate that the child receives all of the attributes, operations, and relationships that are defined in the parent.
- **Interface realization relationships**
In UML diagrams, an interface realization relationship is a specialized type of implementation relationship between a classifier and a provided interface. The interface realization relationship specifies that the realizing classifier must conform to the contract that the provided interface specifies.
- **Instantiation relationships**
In UML diagrams, an instantiation relationship is a type of usage dependency between classifiers that indicates that the operations in one classifier create instances of the other classifier.
- **Package import relationship**
In UML diagrams, a package import relationship allows other namespaces to use unqualified names to refer to package members.
- **Realization relationships**
In UML modeling, a realization relationship is a relationship between two model elements, in which one model element (the client) realizes the behavior that the other model element (the supplier) specifies. Several clients can realize the behavior of a single supplier. You can use realization relationships in class diagrams and component diagrams.
- **Usage relationships**
In UML modeling, a usage relationship is a type of dependency relationship in which one model element (the client) requires another model element (the supplier) for full implementation or operation.

UML Class Diagram

The class diagram depicts a static view of an application. It represents the types of objects residing in the system and the relationships between them. A class consists of its objects, and also it may inherit from other classes. A class diagram is used to visualize, describe, document various different aspects of the system, and also construct executable software code.

It shows the attributes, classes, functions, and relationships to give an overview of the software system. It constitutes class names, attributes, and functions in a separate compartment that helps in software development. Since it is a collection of classes, interfaces, associations, collaborations, and constraints, it is termed as a structural diagram.

Purpose of Class Diagrams

The main purpose of class diagrams is to build a static view of an application. It is the only diagram that is widely used for construction, and it can be mapped with object-oriented languages. It is one of the most popular UML diagrams. Following are the purpose of class diagrams given below:

It analyses and designs a static view of an application.

It describes the major responsibilities of a system.

It is a base for component and deployment diagrams.

It incorporates forward and reverse engineering.

Benefits of Class Diagrams

It can represent the object model for complex systems.

It reduces the maintenance time by providing an overview of how an application is structured before coding.

It provides a general schematic of an application for better understanding.

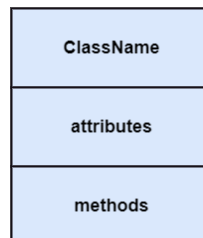
It represents a detailed chart by highlighting the desired code, which is to be programmed.

It is helpful for the stakeholders and the developers.

Vital components of a Class Diagram

The class diagram is made up of three sections:

- **Upper Section:** The upper section encompasses the name of the class. A class is a representation of similar objects that shares the same relationships, attributes, operations, and semantics. Some of the following rules that should be taken into account while representing a class are given below:
 - a. Capitalize the initial letter of the class name.
 - b. Place the class name in the center of the upper section.
 - c. A class name must be written in bold format.
 - d. The name of the abstract class should be written in italics format.
- **Middle Section:** The middle section constitutes the attributes, which describe the quality of the class. The attributes have the following characteristics:
 - The attributes are written along with its visibility factors, which are public (+), private (-), protected (#), and package (~).
 - a. The accessibility of an attribute class is illustrated by the visibility factors.
 - b. A meaningful name should be assigned to the attribute, which will explain its usage inside the class.
- **Lower Section:** The lower section contain methods or operations. The methods are represented in the form of a list, where each method is written in a single line. It demonstrates how a class interacts with data.

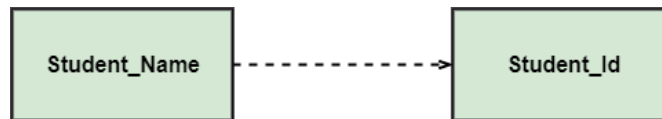


Relationships

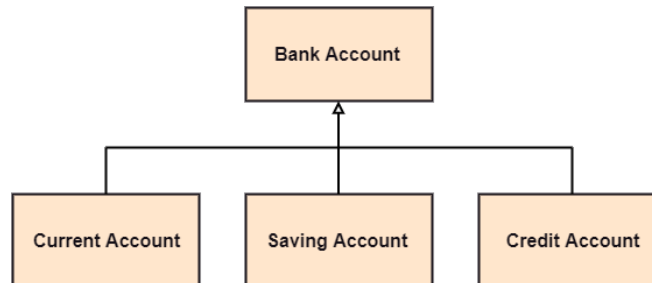
In UML, relationships are of three types:

- **Dependency:** A dependency is a semantic relationship between two or more classes where a change in one class cause changes in another class. It forms a weaker relationship.

In the following example, Student_Name is dependent on the Student_Id.



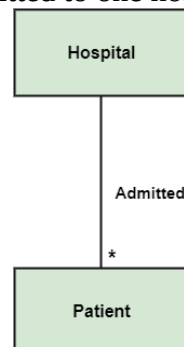
- **Generalization:** A generalization is a relationship between a parent class (superclass) and a child class (subclass). In this, the child class is inherited from the parent class. For example, The Current Account, Saving Account, and Credit Account are the generalized form of Bank Account.



- **Association:** It describes a static or physical connection between two or more objects. It depicts how many objects are there in the relationship. For example, a department is associated with the college.



Multiplicity: It defines a specific range of allowable instances of attributes. In case if a range is not specified, one is considered as a default multiplicity. For example, multiple patients are admitted to one hospital.



Aggregation: An aggregation is a subset of association, which represents has a relationship. It is more specific than association. It defines a part-whole or part-of relationship. In this kind of relationship, the child class can exist independently of its parent class.

The company encompasses a number of employees, and even if one employee resigns, the company still exists.



Composition: The composition is a subset of aggregation. It portrays the dependency between the parent and its child, which means if one part is deleted, then the other part also gets discarded. It represents a whole-part relationship.

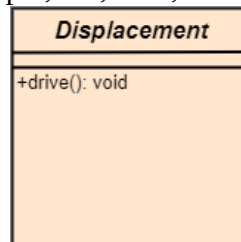
A contact book consists of multiple contacts, and if you delete the contact book, all the contacts will be lost.



Abstract Classes

In the abstract class, no objects can be a direct entity of the abstract class. The abstract class can neither be declared nor be instantiated. It is used to find the functionalities across the classes. The notation of the abstract class is similar to that of class; the only difference is that the name of the class is written in italics. Since it does not involve any implementation for a given function, it is best to use the abstract class with multiple objects.

Let us assume that we have an abstract class named **displacement** with a method declared inside it, and that method will be called as a **drive ()**. Now, this abstract class method can be implemented by any object, for example, car, bike, scooter, cycle, etc.



How to draw a Class Diagram?

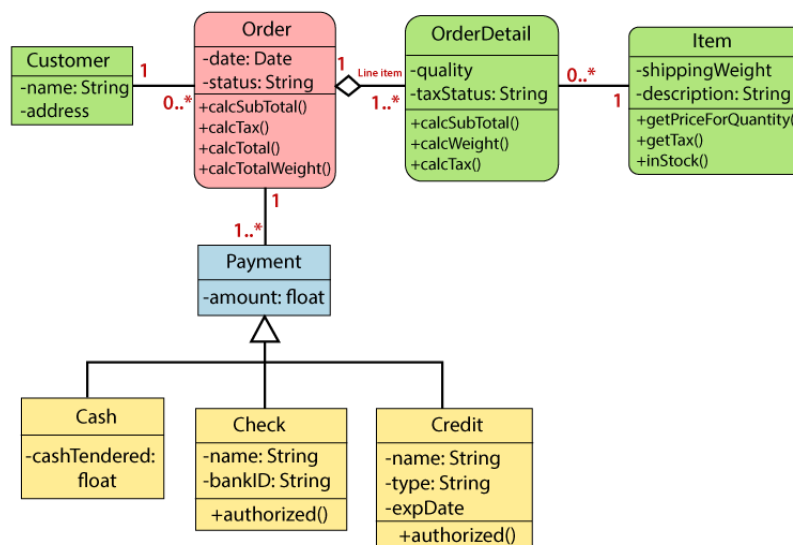
The class diagram is used most widely to construct software applications. It not only represents a static view of the system but also all the major aspects of an application. A collection of class diagrams as a whole represents a system.

Some key points that are needed to keep in mind while drawing a class diagram are given below:

1. To describe a complete aspect of the system, it is suggested to give a meaningful name to the class diagram.
2. The objects and their relationships should be acknowledged in advance.
3. The attributes and methods (responsibilities) of each class must be known.
4. A minimum number of desired properties should be specified as more number of the unwanted property will lead to a complex diagram.
5. Notes can be used as and when required by the developer to describe the aspects of a diagram.
6. The diagrams should be redrawn and reworked as many times to make it correct before producing its final version.

Class Diagram Example

A class diagram describing the sales order system is given below.



Usage of Class diagrams

The class diagram is used to represent a static view of the system. It plays an essential role in the establishment of the component and deployment diagrams. It helps to construct an executable code to perform forward and backward engineering for any system, or we can say it is mainly used for construction. It represents the mapping with object-oriented languages that are C++, Java, etc. Class diagrams can be used for the following purposes:

1. To describe the static view of a system.
2. To show the collaboration among every instance in the static view.
3. To describe the functionalities performed by the system.
4. To construct the software application using object-oriented languages.

Task: Using Star UML, draw the following diagrams (Fig.1, Fig.2, Fig.3)

To do this, you start Star UML. You choose add a model choose a class diagram.

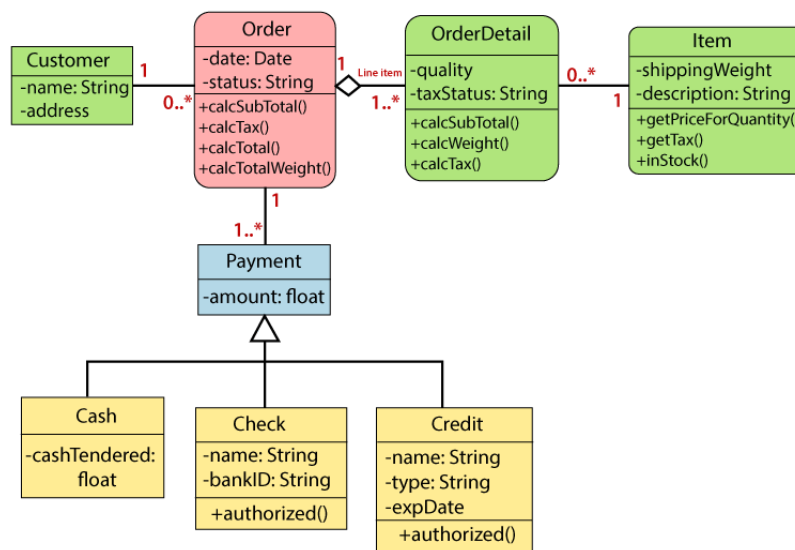


Fig.1

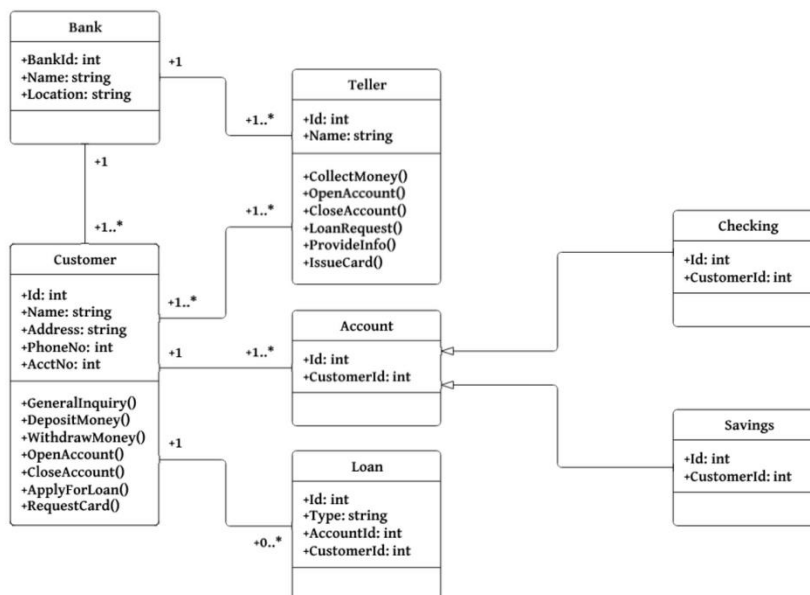


Fig.2 Example of a Class Diagram for a Banking System

The diagram below illustrates aggregation relationships between classes. The lighter aggregation indicates that the class "Account" uses AddressBook, but does not necessarily contain an instance of it. The strong, composite aggregations by the other connectors indicate ownership or containment of the source classes by the target classes, for example Contact and ContactGroup values will be contained in AddressBook.

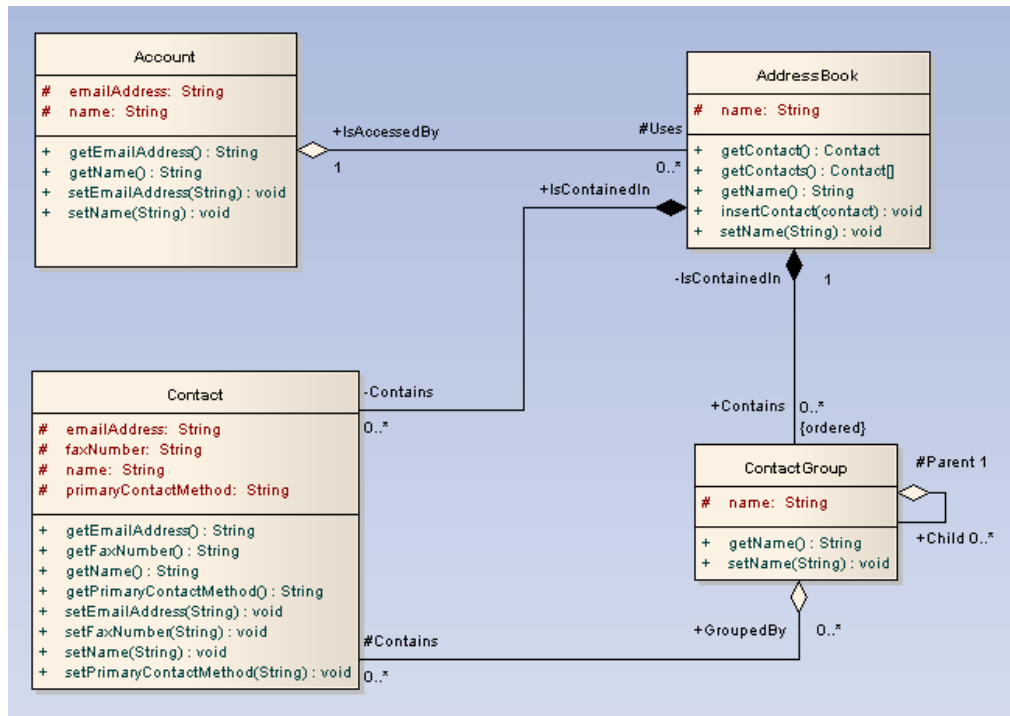


Fig.3

Task 1: Create a class diagram using the following code:

```

using System;
using System.Collections.Generic;

namespace Builder
{
    public class Phone
    {
        public String Model { get; set; }
        public double Price { get; set; }
        public String Color { get; set; }
        public List<String> Extras { get; set; }

        public Phone()
        {
            this.Extras = new List<String>();
        }

        public void About()
        {
            Console.WriteLine("Phone Information");
        }
    }
}

```

```

        Console.WriteLine("\n Model: {0} Color: {1}", this.Model, this.Color);
        Console.Write("\n Extras: ");
        foreach (var extra in Extras)
        {
            Console.Write(extra + " ");
        }

        Console.WriteLine("\n Price: {0}", this.Price);
    }
}

```

```

interface IPhoneBuilder
{
    void SetModel();
    void SetPrice();
    void SetColor();
    void SetExtras();
    Phone GetPhone();
}

```

```

class PhoneCreator
{
    private readonly IPhoneBuilder phoneBuilder;
    public PhoneCreator(IPhoneBuilder builder)
    {
        this.phoneBuilder = builder;
    }

    public void CreatePhone()
    {
        phoneBuilder.SetModel();
        phoneBuilder.SetColor();
        phoneBuilder.SetExtras();
        phoneBuilder.SetPrice();
    }

    public Phone GetPhone()
    {
        return phoneBuilder.GetPhone();
    }
}

```

```

public class ApplePhone : IPhoneBuilder
{
    Phone phone = new Phone();

    public void SetModel()
    {
        phone.Model = "S8";
    }
}

```



```

    }
    public void SetPrice()
    {
        phone.Price = 999.99;
    }
    public void SetColor()
    {
        phone.Color = "White";
    }
    public void SetExtras()
    {
        phone.Extras.Add("Camera");
    }

    public Phone GetPhone()
    {
        return phone;
    }
}

public class SamsungBuilder : IPhoneBuilder
{
    Phone phone = new Phone();

    public void SetModel()
    {
        phone.Model = "X9";
    }
    public void SetPrice()
    {
        phone.Price = 899.99;
    }
    public void SetColor()
    {
        phone.Color = "White/Black";
    }
    public void SetExtras()
    {
        phone.Extras.Add("Camera");
        phone.Extras.Add("Headphone Jack");
    }

    public Phone GetPhone()
    {
        return phone;
    }
}

public class NokiaBuilder : IPhoneBuilder

```

```

{
    Phone phone = new Phone();

    public void SetModel()
    {
        phone.Model = "X5";
    }
    public void SetPrice()
    {
        phone.Price = 100.00;
    }
    public void SetColor()
    {
        phone.Color = "Silver";
    }
    public void SetExtras()
    {
        phone.Extras.Add("Changable Batery");
        phone.Extras.Add("Headphone Jack");
    }

    public Phone GetPhone()
    {
        return phone;
    }
}

class Program
{
    static void Main(string[] args)
    {
        PhoneCreator phoneCreator1 = new PhoneCreator(new ApplePhone());
        phoneCreator1.CreatePhone();
        Phone phone1 = phoneCreator1.GetPhone();
        phone1.About();

        PhoneCreator phoneCreator2 = new PhoneCreator(new SamsungBuilder());
        phoneCreator2.CreatePhone();
        Phone phone2 = phoneCreator2.GetPhone();
        phone2.About();

        PhoneCreator phoneCreator3 = new PhoneCreator(new NokiaBuilder());
        phoneCreator3.CreatePhone();
        Phone phone3 = phoneCreator3.GetPhone();
        phone3.About();

        Console.ReadLine();
    }
}

```

```
}
```

Task 2: Create a class diagram using the following code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Adapter
{
    interface IAirTransport
    {
        void Fly();
    }

    public class Plane : IAirTransport
    {
        public void Fly()
        {
            Console.WriteLine("Plane flies in the sky\n");
        }
    }

    class Pilot
    {
        public void Reallocate(IAirTransport transport)
        {
            transport.Fly();
        }
    }

    interface ISeaTreansport
    {
        void Sail();
    }

    class Ship
    {
        public void Sail()
        {
            Console.WriteLine("The ship sails in the see\n");
        }
    }

    class SeaToAirTransportAdapter : IAirTransport
    {
        private Ship ship;
        public SeaToAirTransportAdapter(Ship ship)
        {
```

```

        this.ship = ship;
    }

    public void Fly()
    {
        this.ship.Sail();
    }
}

class Program
{
    static void Main(string[] args)
    {
        Pilot pilot = new Pilot();
        IAirTransport plane = new Plane();
        pilot.Reallocate(plane);

        Ship ship = new Ship();
        IAirTransport seaTransport = new SeaToAirTransportAdapter(ship);
        pilot.Reallocate(seaTransport);

        Console.ReadLine();
    }
}

```

Task 3: Create a class diagram using the following code:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Command
{
    public interface ICommand
    {
        void Execute();
    }

    public class PowerButton
    {
        private List<ICommand> _commands = new List<ICommand>();

        public void StoreAndExecute(ICommand command)
        {
            _commands.Add(command);
            command.Execute();
        }
    }
}

```

```

public class Computer
{
    public void TurnOn()
    {
        Console.WriteLine("The computer is on");
    }

    public void TurnOff()
    {
        Console.WriteLine("The computer is off");
    }
}

public class StartCommand : ICommand
{
    private Computer _computer;

    public StartCommand(Computer computer)
    {
        _computer = computer;
    }

    public void Execute()
    {
        _computer.TurnOn();
    }
}

public class ShutDownCommand : ICommand
{
    private Computer _computer;

    public ShutDownCommand(Computer computer)
    {
        _computer = computer;
    }

    public void Execute()
    {
        _computer.TurnOff();
    }
}

class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Enter commands (Start\\ShutDown): ");
        string comand = Console.ReadLine();
    }
}

```

```

Computer computer = new Computer();
ICommand start = new StartCommand(computer);
ICommand shutDown = new ShutDownCommand(computer);

PowerButton s = new PowerButton();

if (comand == "Start")
{
    s.StoreAndExecute(start);
}
else if (comand == "ShutDown")
{
    s.StoreAndExecute(shutDown);
}
else
{
    Console.WriteLine("Accepted commands are \"Start\" and \"ShutDown\".");
}

Console.ReadKey();
}
}
}

```

Task 4: Create a class diagram using the following code:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Mediator
{
    abstract class Mediator
    {
        public abstract void Tell(string msg, Person person);
    }

    abstract class Person
    {
        protected Mediator mediator;
        public Person(Mediator mediator)
        {
            this.mediator = mediator;
        }

        public virtual void Tell(string message)
        {

```

```

        mediator.Tell(message, this);
    }

    public abstract void Notify(string message);
}

class Customer : Person
{
    public Customer(Mediator mediator) : base(mediator) { }

    public override void Notify(string message)
    {
        Console.WriteLine("Съобщение на клиента: " + message);
    }
}

class Waitress : Person
{
    public Waitress(Mediator mediator) : base(mediator) { }

    public override void Notify(string message)
    {
        Console.WriteLine("Съобщение на сервитьора: " + message);
    }
}

class Cheff : Person
{
    public Cheff(Mediator mediator) : base(mediator) { }

    public override void Notify(string message)
    {
        Console.WriteLine("Съобщение на готвача: " + message);
    }
}

class ManagerMediator : Mediator
{
    public Person Customer { get; set; }
    public Person Waitress { get; set; }
    public Person Cheff { get; set; }
    public override void Tell(string msg, Person person)
    {
        if (Customer == person)
            Waitress.Notify(msg);
        else if (Waitress == person)
            Cheff.Notify(msg);
        else if (Cheff == person)
            Waitress.Notify(msg);
    }
}

```

```

    }

class Program
{
    static void Main(string[] args)
    {
        ManagerMediator mediator = new ManagerMediator();
        Person customer = new Customer(mediator);
        Person waitress = new Waitress(mediator);
        Person cheff = new Cheff(mediator);

        mediator.Customer = customer;
        mediator.Waitress = waitress;
        mediator.Cheff = cheff;

        customer.Tell("Искам да поръчам");
        waitress.Tell("Направена е поръчка трябва да се сготви");
        cheff.Tell("Поръчката е готова");

        Console.ReadKey();
    }
}

```

Task 5: Create a class diagram using the following code:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Observer
{
    abstract class Computer
    {
        private string _server;
        private int _cpuUsage;
        private List<IAAdministrator> _administrators = new List<IAAdministrator>();
        public Computer(string symbol, int cpuUsage)
        {
            this._server = symbol;
            this._cpuUsage = cpuUsage;
        }

        public void Attach(IAAdministrator administrator)
        {
            _administrators.Add(administrator);
        }

        public void Detach(IAAdministrator administrator)

```



```

    {
        _administrators.Remove(administrator);
    }

    public void Notify()
    {
        foreach (IAdministrator administrator in _administrators)
            administrator.Update(this);

        Console.WriteLine("");
    }

    public int CpuUsage
    {
        get { return _cpuUsage; }
        set
        {
            if (_cpuUsage != value)
            {
                _cpuUsage = value;
                Notify();
            }
        }
    }

    public string ServerName
    {
        get { return _server; }
    }
}

class MainServer : Computer
{
    public MainServer(string symbol, int cpuUsage)
        : base(symbol, cpuUsage)
    {
    }
}

interface IAdministrator
{
    void Update(Computer computer);
}

class Administrator : IAdministrator
{
    private string _name;
    private Computer _computer;
    public Administrator(string name)
    {

```

```

        this._name = name;
    }

    public void Update(Computer computer)
    {
        Console.WriteLine("Notified {0} of cpu usage change of {1} to {2}%", _name,
computer.ServerName, computer.CpuUsage);
    }

    public Computer Computer
    {
        get { return _computer; }
        set { _computer = value; }
    }
}

class Program
{
    static void Main(string[] args)
    {
        MainServer mainServer = new MainServer("Server0", 20);
        MainServer mainServer2 = new MainServer("Server1", 1);

        Administrator admin1 = new Administrator("System Administrator");
        Administrator admin2 = new Administrator("Support Admin");

        mainServer.Attach(admin1);
        mainServer2.Attach(admin2);
        mainServer.Attach(admin1);

        mainServer.CpuUsage = 35;
        mainServer2.CpuUsage = 11;
        mainServer.CpuUsage = 10;
        mainServer.CpuUsage = 89;
        mainServer2.CpuUsage = 43;

        mainServer.Detach(admin2);
        mainServer.CpuUsage = 29;

        Console.ReadKey();
    }
}

```

Task 6: Create a class diagram using the following code:

```

using System;
using System.Collections.Generic;

```

```

namespace Visitor
{

```

```

interface IVisitor
{
    void Visit(Element element);
}

class MatureVisitor : IVisitor
{
    public void Visit(Element element)
    {
        BankAccount bankAccount = element as BankAccount;
        bankAccount.Balance += (bankAccount.Balance * bankAccount.InterestRate);
        Console.WriteLine("{0} {1}'s new balance: {2:C}", bankAccount.GetType().Name,
bankAccount.Name, bankAccount.Balance);
    }
}

class InterestRateVisitor : IVisitor
{
    private double deltaInterestRate;
    public InterestRateVisitor(double DeltaInterestRate)
    {
        this.deltaInterestRate = DeltaInterestRate;
    }

    public void Visit(Element element)
    {
        BankAccount bankAccount = element as BankAccount;
        bankAccount.InterestRate += deltaInterestRate;
        Console.WriteLine("{0} {1}'s new interest rate is {2:C}",
bankAccount.GetType().Name, bankAccount.Name, bankAccount.InterestRate);
    }
}

class TaxVisitor : IVisitor
{
    public void Visit(Element element)
    {
        BankAccount bankAccount = element as BankAccount;
        bankAccount.Balance -= 5.0;
        Console.WriteLine("{0} {1}'s new balance: {2:C}", bankAccount.GetType().Name,
bankAccount.Name, bankAccount.Balance);
    }
}

class BankAccount : Element
{
    private string _name;
    private double _balance;
    private double _interestRate;
    public BankAccount(string name, double balance, double interestRate)

```

```

    {
        this._name = name;
        this._balance = balance;
        this._interestRate = interestRate;
    }

    public string Name
    {
        get { return _name; }
        set { _name = value; }
    }

    public double Balance
    {
        get { return _balance; }
        set { _balance = value; }
    }

    public double InterestRate
    {
        get { return _interestRate; }
        set { _interestRate = value; }
    }

    public override void Accept(IVisitor visitor)
    {
        visitor.Visit(this);
    }
}

abstract class Element
{
    public abstract void Accept(IVisitor visitor);
}

class BankAccounts
{
    private List<BankAccount> _bankAccounts = new List<BankAccount>();
    public void Attach(BankAccount bankAccount)
    {
        _bankAccounts.Add(bankAccount);
    }

    public void Accept(IVisitor visitor)
    {
        foreach (BankAccount e in _bankAccounts)
        {
            e.Accept(visitor);
        }
        Console.WriteLine();
    }
}

```

```

    }
}

class SimpleDeposit : BankAccount
{
    public SimpleDeposit() : base("Deposit Type 1", 500.0, 0.01) { }
}

class NormalDeposit : BankAccount
{
    public NormalDeposit() : base("Deposit Type 2", 78000.0, 0.02) { }
}

class BestBankAccount : BankAccount
{
    public BestBankAccount() : base("GoldenSavingAccount", 800000.0, 0.05) { }
}

class Program
{
    static void Main(string[] args)
    {
        BankAccounts account = new BankAccounts();
        account.Attach(new SimpleDeposit());
        account.Attach(new NormalDeposit());
        account.Attach(new BestBankAccount());

        account.Accept(new InterestRateVisitor(0.02));
        account.Accept(new MatureVisitor());
        account.Accept(new TaxVisitor());

        Console.ReadKey();
    }
}

```

Task 7: Create a class diagram using the following code:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Proxy
{
    public interface IServer
    {
        string GetData();
    }
}

```

```

public class LiveServer : IServer
{
    string Data;
    public LiveServer()
    {
        Console.WriteLine("Live Server is Live");
        Data = "TestData";
    }

    public string GetData()
    {
        return Data;
    }
}

public class ProxyServer : IServer
{
    LiveServer server = new LiveServer();
    public ProxyServer()
    {
        Console.WriteLine("Proxy server is in use");
    }

    public string GetData()
    {
        return server.GetData();
    }
}

class Program
{
    static void Main(string[] args)
    {
        ProxyServer proxyServer = new ProxyServer();
        Console.WriteLine("Data from Proxy Server is {0}", proxyServer.GetData());

        Console.ReadKey();
    }
}

```

Task 8: Create a class diagram using the following code:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

```

```

namespace Template_Method

```

```

{
abstract class SoftwareProject
{
    public void MakeProject()
    {
        GetAssignment();
        Develop();
        CodeReview();
        Test();
        SendProjet();
        GetReward();
    }

    public abstract void GetAssignment();
    public abstract void Develop();

    public virtual void CodeReview() { }

    public virtual void Test() { }

    public virtual void SendProjet()
    {
        Console.WriteLine("Изпращане на направеното задание");
    }

    public abstract void GetReward();
}

class UniversityProject : SoftwareProject
{
    public override void GetAssignment()
    {
        Console.WriteLine("Получаване на задание по определена дисциплина");
    }

    public override void Develop()
    {
        Console.WriteLine("Разработка на задание");
    }

    public override void SendProjet()
    {
        Console.WriteLine("Изпращане на задание или представянето му в
упражнение");
    }

    public override void GetReward()
    {
        Console.WriteLine("Получаване на точки нужни за успешно 'Вземане' на
дисциплината");
    }
}

```

```

    }
}

class WorkProject : SoftwareProject
{
    public override void GetAssignment()
    {
        Console.WriteLine("Получаване на задание от клиент");
    }

    public override void Develop()
    {
        Console.WriteLine("Разделяне на задачи");
        Console.WriteLine("Разработка на задачи");
    }

    public override void CodeReview()
    {
        Console.WriteLine("Извършване на Code Review с цел гарантиране на качеството
на написаният код");
    }

    public override void Test()
    {
        Console.WriteLine("Извършване на тестове");
    }

    public override void GetReward()
    {
        Console.WriteLine("Получаване на заплата\\бонус");
    }
}

class Program
{
    static void Main(string[] args)
    {
        UniversityProject universityProject = new UniversityProject();
        universityProject.MakeProject();

        WorkProject workProject = new WorkProject();
        workProject.MakeProject();

        Console.ReadKey();
    }
}

```

Task 9: Create a class diagram using the following code:


```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Composite
{
    interface IRobot
    {
        void ShowIntelect();
    }

    class BasicRobot : IRobot
    {
        private string name;
        private int intelect;

        public BasicRobot(string name, int intelect)
        {
            this.name = name;
            this.intelect = intelect;
        }

        public void ShowIntelect()
        {
            Console.WriteLine(name + " has intelect of " + intelect);
        }
    }

    class AdvancedRobot : IRobot
    {
        private string name;
        private int intelect;
        private List<IRobot> subordinate = new List<IRobot>();

        public AdvancedRobot(string name, int intelect)
        {
            this.name = name;
            this.intelect = intelect;
        }

        public void ShowIntelect()
        {
            foreach (IRobot robot in subordinate)
            {
                robot.ShowIntelect();
            }

            Console.WriteLine(name + " has intelect of " + intelect);
        }
    }
}

```

```

    }

    public void AddSubordinate(IRobot robot)
    {
        subordinate.Add(robot);
    }
}

class Program
{
    static void Main(string[] args)
    {
        BasicRobot basic1 = new BasicRobot("Model 0", 1);
        BasicRobot basic2 = new BasicRobot("Model 1", 3);
        AdvancedRobot advanced1 = new AdvancedRobot("Model 0+", 11);
        AdvancedRobot advanced2 = new AdvancedRobot("Model x1+", 20);
        AdvancedRobot advanced3 = new AdvancedRobot("AModel 3", 42);

        advanced1.AddSubordinate(basic1);
        advanced2.AddSubordinate(advanced1);
        advanced3.AddSubordinate(basic2);
        advanced3.AddSubordinate(advanced2);

        (advanced3 as IRobot).ShowIntellect();

        Console.ReadKey();
    }
}

```

Task 10: Create a class diagram using the following code:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Decorator
{
    public interface IComputerProgram
    {
        void AddExtra();
    }

    public class BasicProgram : IComputerProgram
    {
        void IComputerProgram.AddExtra()
        {

```

```

        Console.WriteLine("Symple buggy program");
    }
}

public abstract class ExtA : IComputerProgram
{
    protected IComputerProgram input;

    public ExtA(IComputerProgram i)
    {
        input = i;
    }

    void IComputerProgram.AddExtra() { }
}

public class FixBugs : ExtA, IComputerProgram
{
    public FixBugs(IComputerProgram i) : base(i) { }

    void IComputerProgram.AddExtra()
    {
        input.AddExtra();
        Console.WriteLine("Bugs fixed");
    }
}

public class AddComments : ExtA, IComputerProgram
{
    public AddComments(IComputerProgram i) : base(i) { }

    void IComputerProgram.AddExtra()
    {
        input.AddExtra();
        Console.WriteLine("Useful coments added");
    }
}

public class RemoveMagicNumbers : ExtA, IComputerProgram
{
    public RemoveMagicNumbers(IComputerProgram i) : base(i) { }

    void IComputerProgram.AddExtra()
    {
        input.AddExtra();
        Console.WriteLine("Magic Numbers removed");
    }
}

```

```
class Program
{
    static void Main(string[] args)
    {
        IComputerProgram state1 = new BasicProgram();
        IComputerProgram state2 = new FixBugs(state1);
        IComputerProgram state3 = new AddComments(state2);
        IComputerProgram state4 = new RemoveMagicNumbers(state3);
        state4.AddExtra();

        Console.ReadKey();
    }
}
```